MICROSOFT AZURE

Complete Beginner to

Azure Administrator

Hands-On Learning Guide

3 Real-World Projects • Deep Concept Explanations • Interview Q&A • Real Analogies

NO PRIOR AZURE EXPERIENCE NEEDED

Free Resource — Share with anyone learning Cloud

Who is this guide for?

This guide is for anyone — students, IT professionals, or career switchers — who wants to understand Microsoft Azure from the ground up. No cloud experience is assumed. Every concept is explained with simple language, real-world analogies, hands-on project context, and interview preparation questions.

All examples are based on 3 real projects actually built on Azure. By the end of this guide, you will understand how real enterprise cloud infrastructure works.

TABLE OF CONTENTS

INTRODUCTION

What is Cloud Computing?

Starting from absolute zero — no prior knowledge needed

What is Cloud Computing?

Before cloud existed, companies had to buy physical servers, put them in a room (data center), buy cooling equipment, hire people to maintain them, and pay enormous electricity bills — just to run a website or application.

Cloud computing means renting computing resources (servers, storage, networking, databases) over the internet from a provider like Microsoft — and paying only for what you use, just like paying for electricity or water. You don't own the infrastructure; you use it on-demand.

Real-World Analogy:

Think of it like electricity. Before public electricity grids, every factory had to build its own power plant. Now you just plug into the grid and pay for what you use. Cloud computing does the same thing for computing power.

The 3 Cloud Service Models:

Term	Definition
IaaS — Infrastructure as a Service	You get virtual machines, networks, and storage. You manage the operating system, runtime, and everything above. Example: Azure Virtual Machines. You control almost everything but manage more.
PaaS — Platform as a Service	You get a managed platform to deploy your code. Azure manages the OS, runtime, and infrastructure. Example: Azure App Service. You focus only on your application code.
SaaS — Software as a Service	You use a fully managed application over the internet. You manage nothing. Example: Microsoft 365, Gmail. The provider manages everything.

What is Microsoft Azure?

Microsoft Azure is Microsoft's cloud computing platform. It has over 200 services across computing, networking, storage, databases, AI, DevOps, security, and more. Azure has datacenters in over 60 regions worldwide — meaning your applications can run close to your users anywhere on earth.

Azure Global Infrastructure

Azure divides the world into Regions (physical locations with datacenters), Availability Zones (separate datacenters within a region with independent power/cooling), and Geography (a group of regions that preserve data residency — e.g. India Geography contains Central India, South India, West India regions).

Key Azure Terms You Must Know:

Term	Definition
Subscription	A billing and access container in Azure. Everything you create belongs to a subscription. You are billed per subscription.
Resource	Any service you create in Azure — a virtual machine, database, storage account, etc. is called a resource.
Resource Group	A logical container that groups related resources together. Think of it as a folder for your Azure resources.
Region	A geographical location where Azure has datacenters. Example: East US, West Europe, Central India. You choose where your resources are hosted.
Azure Portal	The web-based graphical interface at portal.azure.com to manage all your Azure resources — no coding required for most tasks.
Azure CLI	Command-line interface to manage Azure resources using text commands. Faster and scriptable for automation.
Azure PowerShell	Another command-line tool using PowerShell syntax to manage Azure. Common in Windows environments.

PROJECT 1

Multi-Tier High Availability Infrastructure

Application Gateway + WAF + VMSS + Key Vault + Backup — Enterprise-Grade Architecture

What we built and why it matters:

This project simulates how a real enterprise web application is deployed in production. Every component solves a specific problem. Internet traffic enters only through a protected gateway. Servers automatically scale up and down with demand. Secrets are stored securely. Data is backed up automatically. Access is controlled and monitored. This is not a tutorial — this is how Fortune 500 companies run their applications.

Architecture Overview:

INTERNET
↓ (HTTPS only — all HTTP blocked)
Application Gateway V2 + WAF V2 (Public IP — the ONLY entry point)
↓ (filtered, inspected, load balanced)
Virtual Machine Scale Set (Private subnet — no public IPs!)
↓ (reads secrets at runtime via Managed Identity)
Azure Key Vault (SSL cert + app secrets — private endpoint)
↓ (daily backups)
Recovery Services Vault (Backup + optional Disaster Recovery)

Chapter 1: Resource Groups

What problem does a Resource Group solve?

When you build an application in Azure, you might create 20, 30, or even 100 different resources — virtual machines, load balancers, databases, storage accounts, network rules, etc. Without any organisation, you would have hundreds of resources mixed together with no way to tell which resources belong to which application, how much each application costs, or who should have access to what.

Real-World Analogy:

A Resource Group is like a project folder on your computer. Just as you put all files for a project inside one folder (Word docs, images, spreadsheets), a Resource Group holds all Azure resources that belong to one application or project. When the project ends, you delete the folder — and everything inside goes with it.

What exactly is a Resource Group?

A Resource Group is a logical container in Azure that holds related resources. 'Logical' means it is not a physical thing — it is just a label that groups resources together in Azure's management layer. The resources themselves might be running in different physical datacenters, but they all appear under one Resource Group in the Azure Portal.

Key Rules of Resource Groups:

- Every Azure resource must belong to exactly one Resource Group — you cannot skip this.
- A resource can only be in ONE Resource Group at a time. You can move it, but not be in two simultaneously.
- The Resource Group itself has a region (for storing metadata), but the resources inside can be in any region.
- Deleting a Resource Group deletes ALL resources inside it — permanently. No trash bin.
- You can apply access control (RBAC) and policies at the Resource Group level — all resources inherit them.
- Resources in different Resource Groups can still communicate with each other.

Resource Group — Governance Features:

Term	Definition
Tags	Key-value pairs you attach to resources or resource groups to organise and track them. Example: Environment=Production, CostCenter=Finance, Owner=Abhijeet. Use tags to generate cost reports per team or per environment.
Locks	Protection against accidental changes or deletion. CanNotDelete lock: anyone can read and modify, but nobody can delete. ReadOnly lock: nobody can modify or delete. Even subscription admins cannot override a lock without removing it first.
RBAC at RG level	Assign roles (Owner, Contributor, Reader, custom) to users or groups at the Resource Group level. All resources inside automatically inherit those permissions. No need to set permissions on each resource individually.
Cost Analysis	In Azure Cost Management, you can filter costs by Resource Group to see exactly how much each application or project is costing you each month.

Interview Q: What is a Resource Group in Azure and why do we use it?

Answer: A Resource Group is a logical container that holds related Azure resources. We use it to group resources that belong to the same application so we can manage them together — deploy them together, apply the same access controls, track costs, and delete them all at once when no longer needed. Without Resource Groups, managing hundreds of resources across an Azure subscription would be chaotic.

Interview Q: Can resources in different Resource Groups communicate with each other?

Answer: Yes. Resource Groups are purely a management and billing concept. They do not create any network isolation. Resources in different Resource Groups can communicate

freely as long as the network rules (NSG) allow it. For network isolation, you use Virtual Networks and Subnets, not Resource Groups.

Chapter 2: Virtual Networks (VNet) and Subnets

What is a network and why do we need one in the cloud?

In a traditional office, computers are connected by physical cables and network switches. They can talk to each other because they're on the same local network. When you move to the cloud, your virtual machines (VMs) also need a network to communicate — but there are no physical cables. Azure provides Virtual Networks (VNets) to simulate a private network in the cloud.

Real-World Analogy:

A VNet is like the plumbing system of an apartment building. All apartments (VMs) are connected through shared pipes (the VNet). Subnets are like separate floors — each floor has its own set of pipes. The apartments on the management floor (App Gateway) don't share pipes with the server floor (VMs), keeping things organised and secure.

What is a VNet?

A Virtual Network (VNet) is an isolated private network you create in Azure. Resources inside a VNet can communicate with each other by default using private IP addresses. Traffic within a VNet stays within Azure's infrastructure — it never goes out to the public internet.

When you create a VNet, you define its address space using CIDR notation — for example 10.0.0.0/16. This means the VNet can contain IP addresses from 10.0.0.0 to 10.0.255.255 — that is 65,536 possible IP addresses for your resources.

What is CIDR Notation?

CIDR (Classless Inter-Domain Routing) is a way to define a range of IP addresses. The number after the slash tells you how many addresses are in the range:

Term	Definition
10.0.0.0/8	16,777,214 addresses — huge network, used for entire company
10.0.0.0/16	65,534 addresses — typical for a VNet (used in our project)
10.0.1.0/24	254 addresses — typical for a Subnet
10.0.1.0/28	14 addresses — tiny subnet, for a few services

What are Subnets?

A subnet is a sub-division of a VNet. You carve the VNet address space into smaller ranges. Each subnet gets a range of IPs from the VNet. Resources (VMs, load balancers) are placed inside subnets.

Why divide into subnets? Because it lets you apply different security rules (NSGs) to different parts of your network. For example, your web servers can be in one subnet, your database in another, and you can block direct internet traffic to the database subnet entirely.

Our Project's Subnet Design

GatewaySubnet (10.0.0.0/24): Application Gateway lives here. This subnet has a public IP — it's the only thing touching the internet. WebSubnet (10.0.1.0/24): VMSS instances (web servers) live here. No public IPs — they only receive traffic from the Application Gateway. This design means if a hacker tries to reach your web servers directly, they cannot — there's no public IP to connect to.

Advanced VNet Concepts:

Term	Definition
VNet Peering	Connect two VNets so resources in each can communicate using private IPs, as if they were on the same network. Traffic flows over Microsoft's backbone network, never the internet. Used in hub-spoke architectures where a central hub VNet connects to many spoke VNets.
Service Endpoints	Extends your VNet's identity to Azure PaaS services (Storage, SQL, Key Vault). Traffic from a specific subnet to that service travels over the Azure backbone, not the internet. But the service still has a public endpoint — just restricted to your subnet.
Private Endpoints	Brings an Azure PaaS service completely inside your VNet by giving it a private IP address. No public endpoint exists. The service is completely invisible to the internet. Strongest security option.
Network Virtual Appliance (NVA)	A virtual machine that acts as a firewall, router, or WAN optimizer inside your VNet. Third-party products like Palo Alto, Cisco, Fortinet run as NVAs in Azure.
User Defined Routes (UDR)	Override Azure's default routing rules. Force all outbound internet traffic through a firewall NVA or Azure Firewall before it leaves your network.

Interview Q: What is the difference between a VNet and a Subnet?

Answer: A VNet is the entire private network — like a large block of land (e.g. 10.0.0.0/16 giving 65,536 addresses). A subnet is a smaller parcel carved from that land (e.g. 10.0.1.0/24 giving 254 addresses). Resources are placed in subnets, not directly in the VNet. Subnets let you organise resources and apply different security rules to different parts of your network.

Interview Q: What is VNet Peering and when would you use it?

Answer: VNet Peering connects two VNets so their resources can communicate privately. Use it when you have a hub-and-spoke architecture — for example, a central hub VNet containing shared services (firewall, DNS, monitoring) that multiple spoke VNets (one per application or business unit) need to access. Traffic between peered VNets uses Microsoft's private backbone, not the internet, so it's fast and secure.

Chapter 3: Network Security Groups (NSG)

What is an NSG?

A Network Security Group is a virtual firewall in Azure. It contains a list of security rules. Each rule says: allow or deny traffic that matches specific conditions (source, destination, port, protocol). Azure evaluates these rules in priority order to decide whether to allow or block network traffic.

Real-World Analogy:

An NSG is like a security checkpoint at an office building. The guard (NSG) has a list of rules: VIP guests (rule priority 100) go straight in. Regular visitors (priority 200) need ID. Vendors (priority 300) go to loading dock only. Unknown people with no ID (priority 65500 — default deny) are blocked. The guard checks the first matching rule and stops there.

NSG Rule Properties:

Term	Definition
Priority	A number from 100 to 4096. Lower number = higher priority = checked first. The FIRST rule that matches the traffic wins. Azure stops checking further rules after a match.
Source	Where the traffic is coming from. Can be: Any, a specific IP address, a CIDR range, a Service Tag (like 'Internet' or 'VirtualNetwork'), or an Application Security Group.
Destination	Where the traffic is going. Same options as Source.
Port	The network port. Example: 80 for HTTP, 443 for HTTPS, 22 for SSH, 3389 for RDP. Can be a specific port, a range (8080-8090), or * for any port.
Protocol	TCP, UDP, or Any.
Action	Allow (let traffic through) or Deny (block traffic).

Default Rules (Cannot be deleted):

Every NSG comes with these default rules at the bottom of the list (priority 65000-65500):

- AllowVNetInBound (priority 65000): Allow all traffic from within the VNet — VMs on the same VNet can talk to each other.
- AllowAzureLoadBalancerInBound (priority 65001): Allow health probes from Azure load balancers.
- DenyAllInBound (priority 65500): Block everything else. This is the catch-all deny rule.

> ¡ **Critical Understanding**
>
> Because of the DenyAllInBound default rule at priority 65500, any traffic that doesn't match a custom allow rule is BLOCKED. Your custom rules must explicitly allow what you want. If you forget to add a rule for port 443, HTTPS traffic will be blocked even if everything else looks right.

Service Tags — Smart Shortcuts:

Instead of manually managing long lists of IP addresses for Azure services, use Service Tags. A Service Tag is a label that Azure automatically keeps up-to-date with the correct IP ranges for that service.

Term	Definition
Internet	All public IP addresses on the internet. Use this as source to allow inbound internet traffic or as destination to allow outbound internet access.
VirtualNetwork	All IP address spaces in your VNet (and peered VNets). Use this to allow VMs to talk to each other.
AzureLoadBalancer	Azure's load balancer health probe IPs. Must allow inbound if your VMs are behind a load balancer.
AzureCloud	All Azure datacenter IP ranges. Use to allow traffic from/to Azure services in general.
Storage	IP ranges for Azure Storage service in a specific region. Example: Storage.WestUS2

NSG Placement:

- Subnet-level NSG: Applied to all resources in that subnet. Think of it as the outer fence.
- NIC-level NSG: Applied to a specific VM's network card. Think of it as the individual VM's door lock.
- For inbound traffic: Subnet NSG is checked first, then NIC NSG. BOTH must allow the traffic.
- For outbound traffic: NIC NSG is checked first, then Subnet NSG. BOTH must allow the traffic.

Interview Q: How does NSG rule priority work? Give an example.

Answer: NSG processes rules from the lowest priority number to the highest, and stops at the first match. Example: Rule 100 says Allow HTTPS from Internet to port 443. Rule 200 says Deny everything from Internet. Rule 65500 (default) says Deny All. An HTTPS request from the internet hits rule 100 first — it matches, so it's allowed. Azure never checks rules 200 or 65500. An HTTP request on port 80 doesn't match rule 100, passes rule 200 and is denied there. This way, you can have general deny rules at low priority and specific allow rules at higher priority.

Interview Q: What is the difference between NSG and Azure Firewall?

Answer: NSG is a basic, free layer-4 (port/IP-based) firewall for controlling traffic into and out of subnets and VMs. Azure Firewall is a fully managed, premium firewall service with advanced features: FQDN filtering (allow/deny by domain name like *.google.com), threat intelligence (automatically block known malicious IPs), TLS inspection, intrusion detection, and centralized management across multiple VNets. Use NSG for basic subnet security. Add Azure Firewall for enterprise-grade network security with deep inspection.

Chapter 4: Application Gateway & WAF V2

What problem does Application Gateway solve?

Imagine your website is so popular that one server cannot handle all the traffic. You spin up 5 servers — but how do users connect to all 5? They only know one website address. Someone needs to receive all incoming traffic and distribute it fairly across the 5 servers. That is a load balancer.

But not just any load balancer. Modern web applications need a smart load balancer that understands HTTP and HTTPS — one that can read the URL, inspect headers, terminate SSL, and even block malicious attacks. That is what Application Gateway does.

Real-World Analogy:

Imagine a large hospital reception desk. Every patient walks in through the main entrance. The receptionist (Application Gateway) looks at why each person is there: Emergencies go to the ER ward. Scheduled appointments go to the outpatient ward. Children go to the paediatrics ward. Nobody can sneak past the receptionist into the wards directly. And the receptionist also checks IDs (WAF) to make sure no troublemakers get in.

Application Gateway — Core Components:

Term	Definition
Frontend IP	The public (or private) IP address that clients connect to. This is your website's IP address. Application Gateway V2 supports static public IPs — the IP never changes.
Listener	Listens for incoming connections on a specific IP, port, and protocol. Basic listener: handles one domain. Multi-site listener: handles multiple domains on the same IP (e.g. api.mysite.com and www.mysite.com on the same gateway).
Routing Rule	Maps a listener to a backend pool. Basic routing: all traffic from listener goes to one backend. Path-based routing: /api/* goes to API servers, /images/* goes to image servers, / goes to web servers.
Backend Pool	The group of servers that receive traffic. Can contain: VM IP addresses, VMSS (our project), App Service URLs, or even servers outside Azure.
Backend HTTP Settings	Defines how Application Gateway communicates with backend servers: HTTP or HTTPS, port number, session affinity settings, connection draining time.

Health Probe	Application Gateway regularly sends test requests to backend servers. If a server fails to respond correctly, it is removed from rotation until it recovers. Custom probes can test specific URLs like /health.

SSL Termination — Explained Simply:

HTTPS traffic is encrypted. When it arrives at Application Gateway, the gateway decrypts it (this is called SSL termination). Then it sends the traffic to your backend servers as plain HTTP. This means:

- Your backend servers don't need SSL certificates — they save CPU on decryption.
- Application Gateway holds the SSL certificate (stored in Key Vault in our project).
- Users see the padlock in their browser (HTTPS is secure end-to-end from user to App Gateway).

End-to-end SSL: If you want traffic to also be encrypted between Application Gateway and your backend servers, you can configure end-to-end SSL. App Gateway decrypts, inspects, then re-encrypts before forwarding. More CPU-intensive but maximum security.

What is WAF (Web Application Firewall)?

WAF is a security layer built on top of Application Gateway. It inspects every HTTP/HTTPS request for malicious patterns before the request reaches your servers. It uses the OWASP (Open Web Application Security Project) Core Rule Set — a globally recognised set of rules that detect common web attacks.

OWASP Top 10 — Attacks WAF Blocks:

Term	Definition
SQL Injection	Attacker sends SQL code in a form field hoping it gets executed in your database. Example: entering ' OR '1'='1 in a login field to bypass authentication. WAF detects SQL keywords in unusual places and blocks the request.
Cross-Site Scripting (XSS)	Attacker injects malicious JavaScript code into a web page that gets executed in other users' browsers. WAF detects script tags and event handlers in input fields.
Path Traversal	Attacker tries to access files outside the web root by using ../ in URLs. Example: ../../../../etc/passwd to read the Linux password file. WAF blocks these patterns.
Command Injection	Attacker embeds operating system commands in input fields hoping the server executes them. WAF detects shell command patterns.
Broken Access Control	Attacker accesses resources they shouldn't have access to. WAF combined with proper authentication handles this.

WAF Modes

Detection Mode: WAF monitors and logs all suspicious requests but does NOT block them. Use this when first deploying to understand what traffic your app receives without disrupting

users. Prevention Mode: WAF actively blocks requests that match attack patterns. Use this in production. In our project we used Prevention Mode with OWASP 3.2 ruleset.

Application Gateway SKUs:

Term	Definition
Standard V2	Load balancing only, no WAF. Supports autoscaling, availability zones, static VIP. Use for internal or non-public-facing apps.
WAF V2	Standard V2 PLUS Web Application Firewall. Used in our project. Required for any public-facing web application in production.
V1 (Legacy)	Older version. No autoscaling, no zone redundancy. Microsoft recommends migrating to V2. Do not use for new deployments.

Interview Q: What is the difference between Azure Load Balancer and Application Gateway?

Answer: Azure Load Balancer is a Layer 4 (transport layer) load balancer. It distributes TCP and UDP traffic based on IP address and port number. It cannot read HTTP content. Application Gateway is a Layer 7 (application layer) load balancer. It understands HTTP/HTTPS — it can route based on URL path (/api vs /images), hostname, cookies, headers. It also provides SSL termination and WAF security. Use Load Balancer for non- HTTP workloads (game servers, database replication). Use Application Gateway for web applications.

Interview Q: A client reports their request is being blocked by WAF even though it's a legitimate request. What do you do?

Answer: This is called a false positive. Steps: 1) Check WAF logs in Application Gateway diagnostics to find the specific rule ID that triggered. 2) Review the rule — understand why it matched. 3) Two options: Create a WAF exclusion for the specific request attribute that caused the false positive (e.g. exclude a specific request header from a specific rule), OR create a custom WAF rule to explicitly allow the specific traffic before the OWASP rules are checked. Never disable WAF entirely for false positives — always use targeted exclusions.

Chapter 5: Virtual Machine Scale Sets (VMSS)

What is a Virtual Machine and why isn't one enough?

A Virtual Machine (VM) is a software-based emulation of a physical computer. It runs an operating system (Windows or Linux) and your applications, but it exists as software inside Azure's physical servers. You can create, start, stop, resize, and delete VMs through the Azure Portal.

One VM is often not enough for production. What happens when your app becomes popular and traffic doubles? The single VM gets overloaded and slows down or crashes. What if that VM's physical host has a hardware failure? Your app goes down entirely.

Real-World Analogy:

Imagine a restaurant with only one waiter. On a quiet Tuesday it works fine. But on Saturday night with 200 customers, one waiter cannot cope. The solution: hire more waiters when it gets busy (scale out), send some home when it's quiet (scale in). VMSS automatically does this for your servers — hiring new server instances when traffic spikes and releasing them when traffic drops.

What is VMSS?

Virtual Machine Scale Set (VMSS) is an Azure service that manages a group of identical VMs. All VMs in the set run the same OS image, the same configuration, and the same application. VMSS automatically adds or removes VMs based on rules you define (autoscaling) or a schedule. It distributes traffic across all running VMs via a load balancer (in our case, Application Gateway).

VMSS Key Concepts:

Term	Definition
Instance	Each individual VM inside the scale set. Instances are numbered: instance-0, instance-1, etc.
Capacity	The number of VM instances currently running. VMSS automatically changes capacity based on autoscale rules. You define minimum (e.g. 2) and maximum (e.g. 10) capacity limits.
Scale Out	Adding more VM instances when load increases. Horizontal scaling. This is what VMSS is designed for — add more identical servers.
Scale In	Removing VM instances when load decreases. Saves money — you don't pay for idle servers.
Scale Up	Moving to a larger VM size (more CPU/memory). Vertical scaling. Requires stopping the VM momentarily. VMSS handles scale-out automatically; scale-up typically requires manual intervention.
Autoscale Rules	Conditions that trigger scaling. Example: If average CPU across all instances exceeds 70% for 5 minutes, add 2 more instances. If average CPU drops below 30% for 10 minutes, remove 1 instance.
Cooldown Period	The wait time after a scale event before another scale event can happen. Prevents thrashing — rapidly adding and removing instances that wastes money.
Image	The template used to create all VM instances. Can be a standard Azure Marketplace image (Ubuntu 22.04, Windows Server 2022) or a custom image you prepared with your software pre-installed.

Availability: Fault Domains and Upgrade Domains:

To protect against failures, Azure spreads VMSS instances across multiple physical hosts:

Fault Domain

A fault domain is a set of hardware (servers) that share the same power supply and network switch. If that hardware fails, all VMs on it go down together. By spreading instances across multiple fault domains, one hardware failure only takes down some of your instances, not all. The other instances keep serving traffic.

Upgrade Domain

When Azure or you update the VM configuration (like OS patches or a new app version), Azure updates one upgrade domain at a time. This rolling approach means not all VMs restart simultaneously — some are always available during updates.

Availability Zones

Even stronger protection. Azure regions have multiple physically separate datacenters called Availability Zones, each with independent power, cooling, and networking. By spreading VMSS instances across availability zones, even if an entire datacenter fails, your remaining instances in other zones keep running. Our project used Availability Zones for 99.99% SLA.

Scaling Policies in Detail:

- Metric-based scaling: Scale based on CPU %, memory %, HTTP queue length, custom metrics sent from your app.
- Schedule-based scaling: Scale out at 8 AM weekdays when business starts, scale in at 8 PM. Predictable workloads.
- Predictive autoscaling: Azure uses machine learning to predict traffic patterns and pre-scales BEFORE load hits — so servers are ready when users arrive.

Interview Q: Why use VMSS instead of creating individual VMs manually?

Answer: Creating individual VMs manually means: manually deploying your app to each one, manually monitoring which ones are overloaded, manually starting new VMs when traffic increases, manually deleting VMs when traffic drops, and if a VM crashes, manually detecting it and starting a replacement. VMSS automates ALL of this. It maintains the desired number of healthy instances, automatically replaces failed instances, and scales out and in based on your rules. It also ensures all instances are identical (same image, same config) — no configuration drift.

Interview Q: What is the difference between Availability Sets and Availability Zones?

Answer: Availability Sets spread VMs across fault domains and upgrade domains within a SINGLE datacenter. They protect against hardware failures within one datacenter. SLA: 99.95%. Availability Zones spread VMs across SEPARATE datacenters within the same

region. Each zone has independent power, cooling, and networking. They protect against entire datacenter failures. SLA: 99.99%. Availability Zones are the modern preferred approach. Use Availability Sets only when Availability Zones are not available in a region.

Chapter 6: Azure Key Vault

What problem does Key Vault solve?

Every application needs sensitive information to run: database passwords, API keys, connection strings, SSL certificates, encryption keys. The wrong way to handle these is to put them directly in your code or configuration files — and this is extremely common and dangerous.

When secrets are in code files, they get committed to Git repositories (even private ones get breached), they appear in log files, they are visible to anyone who has code access, and when they expire or need to change, you have to update every place they appear. Azure Key Vault is the correct solution.

Real-World Analogy:

Key Vault is like a bank vault for your application secrets. Your app doesn't carry its own cash (passwords) around — it goes to the vault and checks out what it needs when it needs it. The vault has strict access controls (who can open it), audit logs (who accessed what and when), and automatic renewal of expiring items. If your app code is stolen, the attacker gets nothing — the secrets were never in the code.

What Key Vault Stores:

Term	Definition
Secrets	Any sensitive text value: database connection strings, API keys, passwords, storage access keys. Stored as name-value pairs with versioning. Example: secret name 'DatabasePassword', value 'P@ssw0rd123!'
Keys	Cryptographic keys for encryption and decryption operations. Azure can use Hardware Security Modules (HSMs) to protect keys — meaning the key never exists in software, only in a tamper-proof hardware device.
Certificates	SSL/TLS certificates for securing HTTPS connections. Key Vault can auto-renew certificates from certificate authorities like DigiCert and GlobalSign before they expire — eliminating the risk of forgotten certificate renewals taking down production.

How Apps Access Key Vault — Managed Identity:

The old way: Give the application a username and password to authenticate to Key Vault. Problem: Now you have another secret (the Key Vault password) to protect. This is circular.

The correct way: Use Managed Identity. You give the VM or App Service an identity in Azure Active Directory (Entra ID). Then you grant that identity permission to read secrets from Key Vault. The VM

can now request secrets from Key Vault and Azure automatically handles the authentication — no credentials to store anywhere.

Term	Definition
System-assigned Managed Identity	An identity created automatically for ONE specific Azure resource (e.g. one VM). When that resource is deleted, the identity is deleted too. Tied 1:1 to the resource.
User-assigned Managed Identity	A standalone identity you create separately. Can be assigned to multiple resources. Useful when multiple VMs or App Services need the same Key Vault access — they all use the same identity instead of each having their own.

Key Vault Security Features:

- Soft Delete: Deleted secrets are kept for 90 days in a soft-deleted state. You can recover them. This protects against accidental deletion.
- Purge Protection: Even after soft delete, nobody can permanently purge (destroy) the data until the retention period ends. Protects against malicious insiders.
- Firewall and VNets: Restrict Key Vault access to specific VNets or IP addresses. Combined with Private Endpoint, Key Vault becomes completely private.
- Access Policies vs RBAC: Two permission models. RBAC (newer, recommended) uses Azure roles. Access Policies (older) define permissions per identity per vault. RBAC is preferred for consistency.
- Audit Logs: Every access to Key Vault is logged. You can see exactly who read which secret, at what time, from which IP address.

Interview Q: Why should you never store passwords or API keys in application code or config files?

Answer: Three main reasons: 1) Security: Code files get committed to source control repositories. Even with private repos, breaches happen. Any developer with code access sees all secrets. 2) Rotation: When a secret expires or is compromised, you must update every place it appears and redeploy — hard to track, easy to miss. 3) Audit: You cannot tell who used the secret or when. Key Vault solves all three: secrets are in one secure place with access control, versioning and rotation, and complete audit logging.

Interview Q: What is a Managed Identity and how does it help with Key Vault?

Answer: A Managed Identity is an automatically managed service account in Azure Entra ID that an Azure resource (VM, App Service, Function) can use to authenticate to other Azure services without any credentials in code. For Key Vault: 1) Enable Managed Identity on your VM. 2) Grant that identity 'Key Vault Secrets User' role on your Key Vault. 3) Your app code calls Key Vault SDK — Azure automatically provides a token using the Managed Identity. No passwords, no secrets to manage, no rotation needed. It's the most secure and convenient way to authenticate Azure resources to each other.

Chapter 7: Private Endpoints

The Problem with Public Endpoints:

Azure services like Key Vault, Storage, and SQL Database have public endpoints — internet-reachable URLs. Even with firewall rules restricting which IPs can connect, traffic from your VM to Key Vault travels through the public internet backbone. For high-security environments (banking, healthcare, government), this is unacceptable.

What is a Private Endpoint?

A Private Endpoint is a network interface with a private IP address in your VNet that connects to an Azure PaaS service. When you create a Private Endpoint for Key Vault, your Key Vault gets a private IP (e.g. 10.0.3.5) inside your VNet. Your VMs access Key Vault using this private IP — traffic never leaves the Azure private network.

Real-World Analogy:

Without Private Endpoint: To visit the bank (Key Vault), you walk through public streets (internet) where you could be followed or robbed. With Private Endpoint: The bank opens a private tunnel directly into your building (VNet). You access it internally without stepping outside. Nobody outside your building even knows the bank is accessible from there.

How Private Endpoints Work — Step by Step:

1. You create a Private Endpoint resource in your VNet for the target service (e.g. Key Vault).
2. Azure creates a network interface with a private IP in your chosen subnet.
3. A private DNS zone is needed (e.g. privatelink.vaultcore.azure.net) so when your VM looks up vault.azure.net, it resolves to the private IP instead of the public IP.
4. You can disable the public endpoint on Key Vault entirely — it becomes invisible to the internet.
5. Only resources in your VNet (or peered VNets) can access it.

Private DNS Zone:

Without this, even with a private endpoint, your VM would still resolve the Key Vault URL to its public IP. The Private DNS Zone overrides DNS resolution: when any VM in your VNet asks 'what is the IP of myvault.vault.azure.net?', the private DNS zone answers with the private IP (10.0.3.5) instead of the public IP.

Interview Q: What is the difference between Service Endpoints and Private Endpoints?

Answer: Service Endpoints: Extend your VNet's identity to an Azure service. Traffic from your subnet to the service goes over Azure's backbone (not public internet) and you can restrict the service to only accept traffic from your subnet. BUT: the service still has a public endpoint accessible from the internet with the right credentials. Private Endpoints: Give the Azure service a real private IP inside your VNet. The service becomes completely private — no public endpoint. Traffic never leaves the Azure private network. Private Endpoints are

more secure, support cross-region access, and support on-premises access via VPN/ExpressRoute. Service Endpoints are simpler and free; Private Endpoints have a cost but are required for truly private deployments.

Chapter 8: Recovery Services Vault & Azure Backup

Why Backup Matters:

Data loss can happen from many causes: accidental file deletion, ransomware encrypting your data, software bugs corrupting databases, hardware failure, or a poorly tested deployment overwriting good data. Without backup, data loss is permanent. With backup, you restore from a recent recovery point and continue operations.

Real-World Analogy:

Azure Backup is like car insurance. You hope you'll never need it, but when something goes wrong, it's the only thing standing between you and total loss. The Recovery Services Vault is the safe storage location where your backup data lives — separate from your VMs so that even if your entire resource group is deleted, your backups survive.

What is a Recovery Services Vault?

A Recovery Services Vault is the container that stores backup data and manages backup policies. It supports two types of workloads:

- Azure Backup: Backs up VMs, SQL databases in VMs, Azure Files shares, SAP HANA databases.
- Azure Site Recovery: Replicates VMs continuously to a secondary region for disaster recovery.

Azure Backup — Key Concepts:

Term	Definition
Backup Policy	Defines how often to back up (daily, weekly) and how long to keep backups (7 days, 30 days, 1 year). Example: Take a backup every day at 2 AM and keep it for 30 days.
Recovery Point	A snapshot of a VM or data at a specific point in time. You can restore from any recovery point within your retention period.
RPO (Recovery Point Objective)	The maximum acceptable data loss measured in time. If RPO is 24 hours and you back up daily, you might lose up to 24 hours of data in a worst-case scenario. Shorter RPO = more frequent backups = higher cost.
RTO (Recovery Time Objective)	The maximum acceptable time to restore service after a failure. If RTO is 4 hours, you must be able to restore operations within 4 hours of an incident.
Soft Delete (14 days)	When backup data is deleted, Azure keeps it for 14 more days in a soft-deleted state. This prevents ransomware or malicious admins from destroying backups.

	You must disable this first before deleting a vault — this is exactly what we did in our project.
Geo-Redundant Storage (GRS)	Backup data is stored with GRS by default — it is replicated to a secondary Azure region automatically. Even if an entire region goes down, your backups are safe in the paired region.

Azure Site Recovery (ASR) — Disaster Recovery:

While Azure Backup protects against data loss, Azure Site Recovery (ASR) protects against complete application outage. ASR continuously replicates your VMs to a secondary Azure region. If your primary region has an outage, you initiate a failover — ASR starts your VMs in the secondary region using the most recent replica. When the primary region recovers, you fail back.

Term	Definition
Replication	ASR continuously copies VM disk changes to the secondary region. A new recovery point is created every few minutes.
Failover	Deliberately switching traffic from the failed primary region to the secondary region. Can be planned (for testing or maintenance) or unplanned (in response to an actual disaster).
Failback	After the primary region recovers, replicating changes from secondary back to primary and switching back to the primary region as the active site.
Replication Policy	Defines recovery point frequency (every 5 minutes, 1 hour) and retention. More frequent recovery points = smaller data loss = higher storage and replication cost.

Interview Q: What is the difference between Azure Backup and Azure Site Recovery?

Answer: Azure Backup protects against DATA loss. It takes periodic snapshots you can restore from. If someone accidentally deletes a database or ransomware corrupts files, you restore from the last backup. But restore takes time — RTO could be hours. Azure Site Recovery protects against APPLICATION AVAILABILITY loss. It continuously replicates VMs to another region. If a region goes down, you fail over in minutes — RTO is very low. Use both together: Backup for data protection and accidental deletion recovery, ASR for disaster recovery and business continuity.

Interview Q: In your project, why was it complex to delete the Recovery Services Vault?

Answer: Azure protects backup vaults with multiple safeguards to prevent accidental destruction of backup data. To delete the vault I had to: 1) Disable Soft Delete — otherwise deleted backup items enter a 14-day soft-delete retention period and still count as dependencies. 2) Stop protection and delete data for each backup item (App-VM and Web- VM) — you cannot delete the vault while active backups exist. 3) Remove any Site Recovery replication policies and protected items. 4) Delete private endpoint connections if any. 5) Finally delete the vault itself. This multi-step process is intentional — it forces administrators to consciously acknowledge they are destroying backup data.

Chapter 9: Azure Entra ID, RBAC & Conditional Access

What is Azure Entra ID?

Azure Entra ID (formerly Azure Active Directory or Azure AD) is Microsoft's cloud-based identity and access management service. It is the identity backbone of Azure — every person who logs into the Azure Portal, every application that accesses Azure services, and every automated process that manages Azure resources must authenticate through Entra ID.

Real-World Analogy:

Entra ID is like the HR department and security badge system of a company. The HR department (Entra ID) manages who works here (users), what department they're in (groups), and what their job title is (role). The security badge system enforces who can enter which rooms (access control). Just as the CEO can access the server room but an intern cannot, in Azure the Owner role can manage everything but a Reader can only view.

Core Entra ID Concepts:

Term	Definition
Tenant	A dedicated instance of Entra ID for your organization. When a company signs up for Azure, they get one tenant. All users, groups, applications, and policies exist within this tenant. Tenant ID is a GUID like 72f988bf-86f1-41af-91ab-2d7cd011db47.
User	A person's account in Entra ID. Has a username (email format) and password. Can be a member (employee) or a guest (external partner with B2B access).
Group	A collection of users. Assign permissions to a group instead of each user individually. When someone joins the team, add them to the group — they get all the right permissions automatically.
Service Principal	An identity for an application or automated process to authenticate to Azure. Not a human — it's how your CI/CD pipeline or application authenticates.
Managed Identity	A special Service Principal automatically managed by Azure. No password to rotate. Azure handles authentication automatically. Best for Azure resources authenticating to other Azure services.
App Registration	Register an application in Entra ID so it can use Entra ID for authentication. Used for custom applications that want to support 'Sign in with Microsoft'.

RBAC — Role-Based Access Control:

RBAC is the authorization system in Azure. It determines what authenticated users can DO after they log in. Without RBAC, everyone would either have full admin access or no access — neither is right for a real organization.

RBAC works through three concepts: Role definitions (what actions are allowed), Scope (where the role applies), and Role assignments (who gets which role at which scope).

Built-in Roles:

Term	Definition
Owner	Full access to all resources AND can manage access (assign roles to others). Very powerful — limit to 1-2 people per subscription. Has contributor access plus ability to grant access to others.
Contributor	Full access to create, modify, and delete resources. Cannot manage access (cannot grant roles to others). Use for developers and engineers who manage infrastructure.
Reader	View all resources but cannot make any changes. Good for auditors, managers who need visibility without modification rights.
User Access Administrator	Can manage access (grant roles) but has no permissions on resources themselves. Useful for administrators who manage permissions without touching workloads.
Custom Roles	Define your own role with exactly the permissions you need. Example: 'VM Operator' role that can start/stop VMs but cannot create or delete them. Built from individual Azure operations like 'Microsoft.Compute/virtualMachines/start/action'.

Scope Hierarchy:

Roles can be assigned at different levels of the Azure hierarchy. Permissions assigned at a higher level are inherited by everything below it:

- Management Group → affects all subscriptions in the management group
- Subscription → affects all resource groups and resources in the subscription
- Resource Group → affects all resources in the resource group
- Resource → affects only that specific resource

Principle of Least Privilege

Always assign the minimum permissions needed for a task. If a developer only needs to deploy to one Resource Group, give them Contributor on that Resource Group only — not the entire subscription. If a monitoring tool only needs to read metrics, give it Reader — not Contributor. Excess permissions mean excess risk if an account is compromised.

Conditional Access:

Conditional Access is a policy engine in Entra ID that applies additional requirements based on signals. It implements Zero Trust security: 'Never trust, always verify.' Even if someone has the right password, Conditional Access can block them or require more proof.

Term	Definition
Signals (IF conditions)	User/group (apply only to admins), IP location (from outside corporate network), Device state (unmanaged device), Application (accessing Azure Portal), Sign-in risk (flagged as suspicious).

Controls (THEN requirements)	Grant access with MFA required, Grant access if device is compliant, Block access entirely, Require password change.
MFA (Multi-Factor Authentication)	Requires a second proof of identity beyond password: phone call, SMS code, authenticator app notification, hardware token. Even if a password is stolen, attacker cannot log in without the second factor.
Named Locations	Define trusted IP ranges (like your office network) or countries. Use in Conditional Access: require MFA when signing in from outside named locations.

Interview Q: What is the difference between Authentication and Authorization?

Answer: Authentication (AuthN) answers 'Who are you?' — it verifies identity using credentials (password, MFA). In Azure, Entra ID handles authentication. Authorization (AuthZ) answers 'What are you allowed to do?' — it checks permissions after identity is confirmed. In Azure, RBAC handles authorization. Example: Logging in to the Azure Portal is authentication. Whether you can create VMs after logging in is authorization (depends on your RBAC role).

Interview Q: What is Conditional Access and why is it important?

Answer: Conditional Access is Entra ID's policy engine that applies access controls based on context. It's important because a password alone is not enough security — passwords get stolen, phished, or leaked. With Conditional Access, even a stolen password isn't enough to access resources: the attacker would also need the MFA second factor, need to be on a managed compliant device, or need to be in a trusted location. It's the core implementation of Zero Trust: verify every access request as if it originates from an untrusted network.

Chapter 10: High Availability, SLAs & Load Balancing Patterns

What is High Availability (HA)?

High Availability means designing systems so they continue to function even when individual components fail. No hardware is 100% reliable — servers fail, network cables break, power supplies die. HA design assumes failures WILL happen and ensures the system keeps running anyway.

Azure SLA Comparison:

Architecture	SLA	Max Downtime/Year	Use When
Single VM (Premium SSD)	**99.9%**	8.7 hours	Dev/Test only
VMs in Availability Set	**99.95%**	4.4 hours	Basic production
VMs in Availability Zones	**99.99%**	52 minutes	Critical production
VMSS across Zones (our project)	**99.99%**	52 minutes	Scalable production

App Service (Standard+)	**99.95%**	4.4 hours	Web apps

Azure Load Balancing — Choosing the Right One:

Term	Definition
Azure Load Balancer (Layer 4)	Distributes TCP/UDP traffic based on IP and port. Does not understand HTTP content. Very fast and cheap. Use for: non-HTTP workloads (game servers, FTP, IoT), distributing traffic to VMs in an availability set, internal load balancing between tiers.
Application Gateway (Layer 7)	HTTP/HTTPS load balancer. Understands URLs, headers, cookies. SSL termination. WAF. Path-based and host-based routing. Use for: all public web applications, APIs, microservices with path-based routing.
Azure Front Door (Global Layer 7)	Global HTTP load balancer. Routes users to the nearest healthy backend across multiple regions. Built-in CDN, WAF, SSL termination. Use for: global applications with users worldwide needing the lowest latency.
Traffic Manager (DNS-based)	Routes users at the DNS level to the best endpoint. Works with any protocol (not just HTTP). Routing methods: Priority (primary/failover), Weighted (A/B testing), Performance (nearest), Geographic (compliance). Use for: multi-region failover for any protocol.

PROJECT 2

CI/CD Pipeline — Automated Deployments

GitHub Actions + Azure App Service + 5 Real Failure Scenarios

Chapter 11: Azure App Service

What is Azure App Service?

Azure App Service is a fully managed platform for hosting web applications, REST APIs, and mobile backends. 'Fully managed' means Azure handles the servers, operating system, runtime updates, load balancing, and scaling — you only manage your application code.

Without App Service, to host a web app you'd need to: provision a VM, install an OS, install Node.js or Python or .NET runtime, configure a web server (nginx, Apache), set up SSL, configure autoscaling, set up monitoring... App Service does all of this for you.

Real-World Analogy:

App Service is like a serviced apartment vs a normal apartment. A normal apartment (Virtual Machine) is bare — you furnish it, do all maintenance, fix plumbing yourself. A serviced apartment (App Service) comes fully furnished and maintained — you just bring your suitcase (code) and move in. You pay more per month, but save huge amounts of time and effort.

App Service Plan — The Compute behind App Service:

When you create an App Service, you must place it in an App Service Plan. The Plan defines the underlying compute resources (how many CPUs, how much memory) and the pricing tier (which features are available). Multiple App Services can share one App Service Plan — they share the same compute resources.

Tier	Plans	Cost	Key Features & Limitations
Free	F1	Free	Shared infrastructure, 60 min/day CPU limit, no custom domain, no SSL, no SLA. Dev learning only.
Shared	D1	~$0.013/hr	Shared infrastructure, custom domain supported, no SSL, no SLA.
Basic	B1-B3	From ~$0.018/hr	Dedicated VMs, custom domain + SSL, manual scale to 3 instances max, no autoscale, no slots.
Standard	S1-S3	From ~$0.10/hr	Autoscaling (up to 10 instances), 5 deployment slots, daily backup, Traffic Manager, VNet integration, 99.95% SLA.
Premium V3	P1v3-P3v3	From ~$0.17/hr	More powerful VMs, 20 deployment slots, 30 instances, zone redundancy, enhanced VNet integration.

Isolated V2	I1v2 - I3v2	From ~$0.80/hr	App Service Environment — fully isolated in your own VNet. No shared infrastructure. Maximum security for regulated industries.

Deployment Slots — Zero Downtime Deployments:

A deployment slot is a live copy of your App Service with its own URL. You can have a production slot (yourapp.azurewebsites.net) and a staging slot (yourapp-staging.azurewebsites.net). The workflow:

6. Deploy new version to staging slot. Test it thoroughly.
7. If testing passes: swap staging and production slots with one click.
8. The swap is instant and zero-downtime — Azure just switches the routing.
9. If something is wrong after the swap: swap back in seconds.

† Why Deployment Slots are Powerful

Without slots: deploying means taking the production app down briefly, then deploying, then hoping it works. If it doesn't, you have downtime while you redeploy the old version. With slots: the old version keeps running in production while you deploy and test the new version in staging. The swap is atomic — users experience zero downtime. Rollback is instant.

Interview Q: What is an App Service Plan and how does it affect your app?

Answer: An App Service Plan is the compute foundation for your App Service. It defines the VM size (CPU and memory), the operating system (Windows or Linux), and the pricing tier. The tier determines which features are available: Free tier has no autoscaling, no custom domains, and no SLA. Standard tier adds autoscaling, deployment slots, and 99.95% SLA. Premium tier adds more powerful VMs and more instances. Multiple App Services can share one plan — they share its resources, which is cost-efficient but means one resource-hungry app can starve others.

Chapter 12: GitHub Actions & CI/CD Pipelines

What is CI/CD and why does it matter?

Before CI/CD existed, deploying software was a manual, error-prone, and stressful process. A developer would write code on their laptop for weeks, then a release manager would manually compile everything, copy files to servers, test in production, and hope nothing broke. Bugs slipped through. Deployments took entire weekends. Teams feared release days.

CI/CD (Continuous Integration / Continuous Deployment) automates this entire process. Every code change automatically triggers a pipeline that builds, tests, and deploys the code. The result: smaller changes deployed more frequently with high confidence. Less risk, more speed.

Real-World Analogy:

CI/CD is like a car assembly line. Each car component goes through automated quality checks (tests) before the next step. If any check fails, the line stops before a defective part continues further. The final product is consistently high quality because defects are caught early — not when the customer already has the car.

CI vs CD — The Distinction:

Term	Definition
Continuous Integration (CI)	Every code commit triggers an automatic build and test run. The goal: detect integration problems early. If two developers' changes conflict or introduce bugs, the CI pipeline fails immediately — before the code reaches production. The developer fixes the issue while it's fresh in their mind.
Continuous Delivery (CD)	Extends CI. After tests pass, the code is automatically deployed to a staging environment. A human still approves the final push to production. Good for teams that want automated staging but human oversight for production.
Continuous Deployment	Fully automated — code goes from commit to production automatically with no human approval step, as long as all automated tests pass. Requires very high test coverage and confidence. Used by mature DevOps teams.

GitHub Actions — Core Concepts:

Term	Definition
Workflow	A YAML file in the .github/workflows/ folder of your repository. Defines what automation to run and when. You can have multiple workflow files for different purposes (CI tests, CD deploy, scheduled cleanup).
Trigger (on:)	The event that starts the workflow. Common triggers: push (someone pushes code), pull_request (someone opens a PR), schedule (cron schedule like every night at midnight), workflow_dispatch (manual trigger from the GitHub UI).
Job	A unit of work within a workflow. Each job runs on its own fresh virtual machine (runner). Jobs run in parallel by default. Use 'needs:' to create dependencies between jobs (job B only starts after job A finishes successfully).
Step	An individual task within a job. Each step either runs a shell command (run: npm install) or uses a pre-built Action (uses: actions/checkout@v3).
Action	A pre-built, reusable piece of automation. The GitHub Marketplace has thousands. Examples: actions/checkout (downloads your code), actions/setup-node (installs Node.js), azure/webapps-deploy (deploys to Azure App Service).
Runner	The machine that executes a job. GitHub provides hosted runners: ubuntu-latest (Linux VM), windows-latest, macos-latest. Each job gets a fresh, clean machine.
Secrets	Encrypted values stored in GitHub repository settings. Referenced in workflows as ${{ secrets.MY_SECRET }}. GitHub masks them in logs so they never appear in plain text. Store Azure credentials, publish profiles, API keys here.

Artifact	Files produced by one job that need to be passed to another job. Use actions/ upload- artifact to save files and actions/download-artifact to retrieve them. Since each job runs on a different machine, you cannot just reference files between jobs without artifacts.

Environment	A named deployment target in GitHub with its own protection rules. Example: 'production' environment requires two reviewers to approve before the deployment job runs.

Our Pipeline Structure:

TRIGGER: Push to main branch

JOB 1: Build and Test (CI)
Step 1: Checkout code (download repo to runner machine)
Step 2: Setup Node.js 18 (install runtime)
Step 3: npm install (install all dependencies)
Step 4: npm test (run automated tests)
Step 5: Upload artifact (save built code for CD job)

JOB 2: Deploy (CD) — only runs if Job 1 PASSES
Step 1: Download artifact (get code from CI job)
Step 2: Deploy to Azure App Service (send code to Azure)
Step 3: Health check (call /health endpoint, fail if not 200 OK)

Interview Q: Why should the deploy job only run after the build and test job passes?

Answer: Because deploying broken code to production is worse than not deploying at all. If tests catch a bug, we want to stop there and fix it. The 'needs: build-and-test' dependency creates a gate: Job 2 only starts if Job 1 completes with a green tick. If Job 1 fails for any reason (syntax error, test failure, build error), Job 2 is automatically cancelled. This ensures only tested, working code reaches Azure.

Interview Q: Why did you add a health check after deployment?

Answer: A pipeline can show 'deployment succeeded' even when the app is crashing. The deployment step only confirms that the ZIP file was uploaded and extracted to Azure App Service — it doesn't know if the app actually started successfully. Our health check step waits 40 seconds then sends an HTTP request to the /health endpoint. If the response is not 200 OK, the pipeline fails and the team knows immediately. Without this, broken deployments go undetected until a user reports an error.

Chapter 13: Service Principals & Managed Identity

Why Can't You Use Your Own Account in a Pipeline?

When a pipeline runs, it needs to authenticate to Azure to deploy resources. You could use your own username and password — but this is terrible practice: if you leave the company, the pipeline breaks. Your account has permissions you don't want automated systems to have. Your password changes and breaks the pipeline. Personal accounts should not be used for automated systems.

Instead, you create a Service Principal — a dedicated identity for the pipeline with specific, limited permissions.

Service Principal vs Managed Identity — Comparison:

Aspect	Service Principal	Managed Identity
Credentials	Client Secret (password) or Certificate — YOU manage it	Automatically managed by Azure — no credentials
Rotation	Must rotate before expiry or pipeline breaks	Azure rotates automatically — nothing to do
Where to use	External systems (GitHub Actions, Jenkins, Terraform)	Azure resources (VM, App Service, Functions)
Security risk	Secret can be stolen if not stored carefully	No secret exists — cannot be stolen
Our Project 2 use	GitHub Actions uses Service Principal to authenticate to Azure	VMSS uses Managed Identity to access Key Vault

Chapter 14: Pipeline Debugging — 5 Real Failure Scenarios

> ¡ **Real Production Scenarios**
>
> These 5 failures were intentionally introduced in our project to build debugging skills. In production, these exact same issues happen. Knowing how to diagnose and fix them is a core DevOps skill that interviewers test for.

Failure 1: Missing Dependency

Scenario: Developer adds a new npm package to code but forgets to run npm install and commit the updated package-lock.json.

What you see in pipeline: npm ci fails with 'Cannot resolve dependency'. Pipeline stops at the Install Dependencies step.

Why this happens: npm ci (which pipelines use instead of npm install) requires an exact package-lock.json. If the file is missing or outdated, it fails.

Fix: Run npm install locally to generate/update package-lock.json. Commit both package.json and package-lock.json to Git. Push again.

Lesson: Always commit package-lock.json. Never .gitignore it.

Failure 2: Expired or Incorrect Credentials

Scenario: The AZURE_CREDENTIALS secret in GitHub was updated with wrong values, or the Service Principal client secret expired.

What you see: The 'Login to Azure' step fails with 'The client does not have authorization' or 'Invalid client secret'.

Why this happens: Service Principal secrets have an expiry date (default 1 year). After expiry, authentication fails.

Fix: Generate a new Service Principal credential using Azure CLI. Update the AZURE_CREDENTIALS secret in GitHub repository settings. Re-run the failed pipeline.

Lesson: Monitor Service Principal expiry dates. Set calendar reminders 30 days before expiry. Consider using Workload Identity Federation (no secrets needed) for GitHub Actions — the modern approach.

Failure 3: App Deploys But Crashes (Silent Failure)

Scenario: Developer introduces a runtime error (referencing a null variable, unhandled exception). The syntax is valid so the pipeline doesn't catch it during build.

What you see: Pipeline shows green. But users get 500 errors or the app returns 503 Service Unavailable.

Why this happens: A successful deployment only means the files were uploaded. It doesn't mean the app runs correctly. Runtime errors only appear when the app actually executes the broken code.

Fix: Check App Service Log Stream (real-time) to see the crash error. Fix the bug. Push again. The health check step in the pipeline catches this before it's considered 'successful'.

Lesson: ALWAYS add a post-deployment health check in your pipeline. Green deployment != working app.

Failure 4: Missing Environment Variable

Scenario: App requires an environment variable (like DATABASE_URL or API_KEY) that exists in the team's local setup but was never added to Azure App Service Configuration.

What you see: App starts, then immediately crashes. Log Stream shows 'FATAL: Required environment variable DATABASE_URL is not set'.

Why this happens: Environment variables on a developer's laptop don't automatically appear in Azure. They must be explicitly added to App Service → Configuration → Application Settings.

Fix: Go to Azure Portal → App Service → Configuration → Application Settings → Add the missing variable → Save (app restarts automatically).

Lesson: Document all required environment variables. Use Key Vault references in App Settings for sensitive values. Check all env vars are present as part of your deployment checklist.

Failure 5: Merge Conflict

Scenario: Two developers edit the same file on different branches. When trying to merge, Git detects conflicting changes.

What you see: Pull Request shows 'This branch has conflicts that must be resolved'. Pipeline is blocked.

Why this happens: Git cannot automatically choose between two different changes to the same lines of code.

Fix: Open the conflicting file. Git marks conflicts with <<<<<<< (your changes), ======= (separator), >>>>>>> (incoming changes). Read both versions, decide which to keep (or combine them), delete the conflict markers, save, commit.

Lesson: Communicate with teammates about what files you're editing. Use short-lived feature branches. Merge from main frequently to stay current. Smaller, more frequent commits have fewer conflicts.

Chapter 15: Monitoring — Logs, Metrics & Alerts

The Three Pillars of Observability:

Term	Definition
Logs	Detailed records of events that happened in your system. 'User John logged in at 14:32:01. API call to /payments failed with 503. VM restarted due to OOM.' Logs are for diagnosing specific incidents after they happen.
Metrics	Numerical measurements over time. CPU is at 72%. Response time is 340ms. Error rate is 0.1%. Metrics are for monitoring trends and triggering alerts when thresholds are breached.
Traces	Following a single user request as it travels through multiple services. 'Request A went: Web → API → Database → Cache → Response in 423ms'. Traces are for understanding distributed system performance.

Azure Monitor — Central Monitoring Hub:

Azure Monitor is the umbrella service that collects, analyzes, and acts on telemetry from all Azure resources. Think of it as Mission Control for your Azure environment.

Term	Definition
Metrics	Auto-collected numerical data from Azure services. CPU %, disk IOPS, HTTP request count, response time. Stored for 93 days. View in Metrics Explorer with charts and graphs.
Logs (Log Analytics)	Structured log data from resources, sent to a Log Analytics Workspace. Queried using KQL (Kusto Query Language). Store logs for months/years for compliance and trend analysis.
Diagnostic Settings	Configure each Azure resource to send its logs and metrics to Log Analytics Workspace, Event Hub (for streaming), or Storage Account (long-term archival).
Alert Rules	Watch a metric or log query and trigger an Action Group when a condition is met. Example: trigger an alert when Http5xx errors exceed 10 per minute for 5 minutes.
Action Groups	Define what happens when an alert fires. Send email, SMS, make a voice call, trigger a webhook, create an ITSM (ServiceNow) ticket, or run an Azure Function or Logic App.
Dashboards	Customisable views pinning metrics charts, log query results, and resource health indicators for at-a-glance visibility.

App Service Specific Monitoring:

- Log Stream: Real-time stdout/stderr from your app. Best for active debugging — watch logs as requests come in.
- App Service Logs: Configure to capture application logs to file system or Blob Storage. Enable 'Application Logging' and set level (Error, Warning, Information, Verbose).

- Kudu (Advanced Tools): Access at yourapp.scm.azurewebsites.net. Provides bash console, file manager, log files, process explorer, and deployment history.
- Application Insights: Deep APM telemetry — request rates, failure rates, response times, exception details, dependency tracking, user flows, live metrics stream.

Interview Q: How do you investigate an App Service returning HTTP 500 errors?

Answer: Systematic approach: Step 1 — Check App Service Overview to confirm the app is running (not stopped). Step 2 — Go to Log Stream (left menu) and watch for error output while reproducing the issue. Step 3 — Check Application Insights Failures blade if Application Insights is configured — it shows failed request details and full exception stack traces. Step 4 — Go to Kudu → LogFiles → Application for stored log files. Step 5 — Check Configuration to verify all required Application Settings (environment variables) are present. Step 6 — If a recent deployment caused the issue, use Deployment Slots to swap back to the previous version while you investigate.

PROJECT 3

Azure App Services Deep Dive

Scaling, Deployment Methods, Custom Domains, SSL & Application Insights

Chapter 16: Scaling, Deployment Methods & Deployment Slots

Understanding App Service Scaling in Detail:

Scaling is how you handle varying amounts of traffic. A well-designed app scales automatically — adding capacity when users arrive and releasing it when they leave. Understanding the options helps you design cost-effective, reliable systems.

Scale Up vs Scale Out — Deep Comparison:

Aspect	Scale Up (Vertical)	Scale Out (Horizontal)
What it means	Move to a larger, more powerful machine (more CPU/RAM)	Add more instances of the same size
Downtime?	Brief restart required when changing plan tier	Zero downtime — new instances added without stopping old ones
Cost model	Linear — bigger machine costs proportionally more	Flexible — pay per instance, remove when not needed
Limit	Maximum VM size available in the region	Up to 30 instances on Premium V3, 10 on Standard
Best for	Apps that cannot be distributed (legacy stateful apps)	Web apps, APIs, stateless microservices
Example	B1 (1 core, 1.75 GB) → B2 (2 core, 3.5 GB)	1 instance → 5 instances during traffic spike → back to 1

Autoscale Rules — How to Configure:

Autoscale watches a metric over a time window. When the metric crosses a threshold consistently, autoscale fires.

🎞 Example Autoscale Rule

Scale Out rule: When average CPU across all instances > 70% for 5 consecutive minutes → Add 2 instances. (Cooldown: 5 minutes before another scale-out can happen.) Scale In rule: When average CPU < 30% for 10 consecutive minutes → Remove 1 instance. (Cooldown: 10 minutes — use a longer cooldown for scale-in to prevent premature removal.) Minimum: 2 instances always running (for HA). Maximum: 10 instances (cost cap).

All Deployment Methods for App Service:

Term	Definition
GitHub Actions	Our CI/CD method from Project 2. Code push → pipeline runs automatically → deployed to Azure. Best for teams using GitHub.
Azure DevOps Pipelines	Microsoft's enterprise CI/CD platform. Deeper integration with Azure services, enterprise features like artifact feeds, test plans, boards. Best for larger organisations.
Deployment Center	Simplified setup in Azure Portal. Connect a GitHub/Azure DevOps/Bitbucket repo and branch. Azure creates the pipeline file automatically. Easiest setup for beginners.
Local Git Deploy	App Service provides a Git remote URL. You push code with 'git push azure main' from your terminal. Kudu receives it and deploys. Good for solo developers doing quick deployments.
ZIP Deploy	Create a ZIP of your app, upload via Azure CLI (az webapp deploy --src-path app.zip) or REST API. Pipeline-friendly. Fast.
FTP/FTPS	Upload files directly via FTP client. Credentials in App Service → Deployment credentials. Old method, no version control, not recommended for production use.
Run From Package	Best practice for production. Deploy a ZIP file and App Service runs the app directly from the mounted ZIP without extracting. Benefits: faster cold starts, files are read-only (prevents drift), deployment is atomic (all files update together).
External Git	Point to any public Git repository URL. Azure pulls code on demand. Great for deploying from open-source repositories without GitHub account setup.

Chapter 17: Custom Domains, SSL/TLS & Application Insights

Custom Domains:

By default, your App Service is accessible at yourapp.azurewebsites.net. For a real business, you need your own domain like www.yourcompany.com. Adding a custom domain to App Service involves DNS configuration to prove you own the domain and point it to your app.

Custom Domain Setup Steps:

10. Purchase a domain from a registrar (GoDaddy, Namecheap, Google Domains, Azure itself).
11. In Azure Portal → App Service → Custom Domains → Add custom domain.
12. Azure shows you what DNS records to add for verification.
13. At your registrar: Add a CNAME record: www.yourcompany.com → yourapp.azurewebsites.net (for subdomains). For root domain (yourcompany.com without www): Add an A record pointing to App Service's IP + a TXT record for verification.
14. Return to Azure Portal and click Validate — Azure verifies the DNS records.
15. After validation: your custom domain is added. Now secure it with SSL.

SSL/TLS Certificate Options:

Term	Definition
App Service Managed Certificate (Free)	Azure automatically provides and renews a free SSL certificate for your custom domain. Zero cost, zero management. Limitation: only for subdomains (www.yourcompany.com), not root domains (yourcompany.com) in all configurations. Also cannot be exported.
App Service Certificate	Purchase through Azure Portal. Azure manages it and automatically stores it in Key Vault and binds it to your App Service. Auto-renews. Costs ~$70-300/year depending on certificate type.
Bring Your Own Certificate (BYOC)	Upload a PFX file (certificate + private key) you purchased from any certificate authority. Must be at least 2048-bit key, must include full chain. Useful if your company already has certificates.
Key Vault Certificate	Reference a certificate stored in Azure Key Vault directly. Best practice — certificate managed centrally, rotated automatically, used by multiple resources.

HTTPS Only Setting

Enable 'HTTPS Only' in App Service → TLS/SSL Settings. This redirects all HTTP traffic (port 80) to HTTPS (port 443) automatically. Users who type http:// are automatically redirected to https://. Never run production web apps without this enabled.

Application Insights — Deep Application Monitoring:

Application Insights is Azure's Application Performance Monitoring (APM) service. While Azure Monitor watches infrastructure health, Application Insights goes inside your application and tracks every request, every error, every database call, and every user interaction.

Application Insights Key Features:

Term	Definition
Live Metrics Stream	Real-time dashboard showing requests/second, failure rate, response time, CPU, and memory — updating every second. Perfect for watching a deployment go out or diagnosing a live incident.
Application Map	Visual diagram of your application's components and how they connect. Web app → Database, Web app → External API, etc. Each connection shows average response time and failure rate. Instantly see which component is causing problems.
Failures Blade	Grouped view of all failed requests and exceptions. Click any failure to see the full stack trace, the request details, and related log entries. Finding the root cause of a bug takes minutes instead of hours.
Performance Blade	Shows the slowest operations ranked by response time. Drill into any operation to see a sample timeline — how long each step took (rendering, database query, external API call).
Availability Tests	Configure URL ping tests that run every 5 minutes from Azure datacenters in multiple regions worldwide. If your app goes down in one region, you know within 5 minutes. Alerts fire automatically.
User Flows	Visual map of how users navigate through your web application — which pages they visit in sequence, where they drop off. Valuable for product improvement.

Log Analytics (KQL)	All Application Insights data is stored in Log Analytics. Write KQL queries to find specific error patterns, unusual traffic, performance degradation over time, or usage trends.

KQL — Kusto Query Language Basics:

KQL is used to query all Azure Monitor and Application Insights data. Knowing basic KQL is essential for Azure administrators.

```
// Find all requests that failed in the last 24 hours:
requests | where success == false | where timestamp > ago(24h) |
summarize count() by resultCode


// Average response time per operation in last hour:
requests | where timestamp > ago(1h) | summarize avg(duration) by name
| order by avg_duration desc


// Count VM restarts in last 7 days:
AzureActivity | where OperationName == 'Restart Virtual Machine' |
where TimeGenerated > ago(7d) | project TimeGenerated, Caller,
ResourceGroup
```

Interview Q: What is Application Insights and what types of problems does it help diagnose?

Answer: Application Insights is Azure's APM (Application Performance Monitoring) service that provides telemetry from inside your application. It helps diagnose: 1) Performance problems — which API endpoints are slow and why (database query taking too long, external API not responding). 2) Errors and exceptions — full stack traces for every unhandled exception in your app. 3) Availability — whether your app is reachable from different geographic regions. 4) Usage patterns — which features users actually use, where they drop off in a workflow. 5) Dependency tracking — how long calls to databases, external APIs, and other services take. Unlike infrastructure monitoring (Azure Monitor), Application Insights understands your application's code-level behaviour.

BONUS

Azure General Topics & Quick Reference

Storage, Networking, Governance, Cost Management — Everything Else You Need

Azure Storage — Complete Overview

4 Types of Azure Storage:

Term	Definition
Blob Storage	Object storage for unstructured data — images, videos, documents, backups, log files, anything. Organized into containers (like folders) and blobs (files). Massively scalable. Three access tiers: Hot (frequent access, higher cost), Cool (infrequent, lower cost), Archive (rarely accessed, lowest cost but hours to retrieve).
File Storage	Fully managed file shares accessible via SMB (Windows file sharing protocol) or NFS. Lift-and-shift replacement for on-premises Windows file servers. Can be mounted as a drive letter on Windows VMs (Z:\) or mounted on Linux VMs. Ideal for shared configuration files, application data shared across multiple VMs.
Queue Storage	Message queue service for decoupling application components. One service puts messages in the queue, another reads and processes them. If the processing service goes down, messages wait in the queue and are processed when it recovers. Maximum message size: 64KB. Perfect for async workflows.
Table Storage	NoSQL key-value store for structured data that doesn't need complex SQL queries. Very cheap, massively scalable. Good for IoT telemetry, user session data, application diagnostics. Not recommended for complex relational data — use SQL Database for that.

Storage Redundancy Options:

Type	Where Copies Are Stored	# Copies	Best For
LRS — Locally Redundant	3 copies in one datacenter, same rack area	3	Dev/Test. Lowest cost. Not for production.
ZRS — Zone Redundant	3 copies across 3 Availability Zones in one region	3	Production workloads needing zone failure protection.
GRS — Geo Redundant	LRS in primary region + LRS copy in paired region	6	Production needing region-level disaster recovery.
GZRS — Geo-Zone Redundant	ZRS in primary + LRS in secondary region	6	Maximum durability and availability. Business critical data.
RA-GRS / RA-GZRS	GRS/GZRS + read access to secondary region	6	When you need to read from secondary even during outage.

Azure Networking — Essential Services

Term	Definition
Azure Bastion	Managed RDP/SSH service. Access VMs directly in your browser through Azure Portal without opening port 3389 (RDP) or 22 (SSH) to the internet. Deploy Bastion into a subnet called AzureBastionSubnet in your VNet. VMs need no public IP. Eliminates the most common attack vector for Azure VMs.
Azure Firewall	Cloud-native, managed stateful firewall. Inspect traffic with FQDN filtering (allow/deny by domain name), threat intelligence feed (automatically block known malicious IPs), IDPS (intrusion detection and prevention). Centrally manage firewall rules across all your VNets from one policy. More expensive than NSG but enterprise-grade.
Azure DDoS Protection	Basic (free, automatic): protects against volumetric attacks on all Azure public IPs. Standard (paid): enhanced protection, attack analytics, real-time metrics during attacks, post-attack reports, and guaranteed SLA. Applied at the VNet level.
Azure DNS	Host your domain's DNS zone in Azure. Manage DNS records (A, CNAME, MX, TXT) with the same Azure tools, RBAC, and APIs as all other resources. Private DNS Zones: DNS resolution that only works within your VNet — used with Private Endpoints.
VPN Gateway	Encrypted tunnel connecting your on-premises network to Azure VNet over the internet. Site-to-site VPN (company office to Azure), Point-to-site VPN (individual device to Azure). Slower than ExpressRoute but cheaper.
ExpressRoute	Dedicated private connection from your datacenter to Azure — not over the internet. Higher bandwidth (up to 100 Gbps), lower latency, higher reliability than VPN. Required for high-security/high-performance hybrid scenarios.

Azure Governance

Term	Definition
Azure Policy	Define, assign, and enforce rules across your Azure environment. Audit mode: report non-compliant resources without blocking. Deny mode: block resource creation if it violates the policy. Example policies: 'All VMs must use Approved VM sizes', 'All resources must have a CostCenter tag', 'Storage accounts must use HTTPS only', 'Resources can only be deployed to approved regions'.
Blueprints (deprecated → Deployment Stacks)	Package policies, RBAC, and resource templates together. Deploy a complete, compliant environment in one step. Useful for ensuring every new subscription starts with the right guardrails.
Management Groups	Organize multiple subscriptions into a hierarchy. Apply policies and RBAC at the management group level — all child subscriptions inherit them. Example hierarchy: Root → Production MG → (Sub-A, Sub-B), Development MG → (Sub-C), Testing MG → (Sub-D). Max 6 levels deep.
Azure Cost Management	Track, analyse, and optimise Azure spending. Set budgets with alerts (email when spending hits 80% of budget). Cost analysis: filter by subscription, resource group, resource, tag. Recommendations for cost savings via Advisor.
Azure Advisor	Free personalised recommendations in 5 areas: Cost (resize/delete underutilised resources), Security (enable MFA, remediate vulnerabilities), Reliability (add backups, availability zones), Performance (enable caching, CDN), Operational Excellence (update deprecated APIs, follow best practices).

Cost Optimisation Strategies

- Reserved Instances: Commit to 1 or 3 years and save up to 72% compared to pay-as-you-go. Best for predictable, always-on workloads (production database servers, always-on web servers).
- Azure Hybrid Benefit: Use your existing on-premises Windows Server and SQL Server licences on Azure VMs. Saves up to 40-85% on Windows VM costs.
- Spot VMs: Use Azure's spare capacity at up to 90% discount. Can be evicted with 30 seconds notice. Best for fault-tolerant, interruptible workloads (batch processing, dev/test, rendering).
- Auto-shutdown: Automatically shut down dev/test VMs at end of day. Save 60%+ on non-production VMs that don't need to run 24/7.
- Right-sizing: Use Azure Advisor to identify oversized VMs. A VM using 5% average CPU might be able to run on a smaller SKU at lower cost.
- Storage lifecycle policies: Automatically move Blob Storage data from Hot to Cool to Archive tier as it ages. Data accessed frequently stays in Hot, rarely accessed data moves to Archive.

Quick Reference — Most Common Interview Questions

Interview Q: What is the difference between IaaS, PaaS, and SaaS? Give Azure examples.

Answer: IaaS (Infrastructure as a Service): You manage OS and above, Azure manages hardware and hypervisor. Azure examples: Virtual Machines, Virtual Networks, Storage. PaaS (Platform as a Service): You manage only your application code and data, Azure manages OS, runtime, middleware. Azure examples: App Service, Azure SQL Database, Azure Functions, AKS. SaaS (Software as a Service): You use a finished application, manage nothing. Azure/Microsoft examples: Microsoft 365, Dynamics 365, Teams. Rule of thumb: IaaS = most control, most responsibility. SaaS = least control, least responsibility.

Interview Q: What is the difference between a VNet and a Subnet?

Answer: A VNet is the entire private network (e.g. 10.0.0.0/16 — 65,534 addresses). A Subnet is a smaller division within the VNet (e.g. 10.0.1.0/24 — 254 addresses). Resources like VMs are placed in subnets. Subnets let you organise resources logically and apply different security rules (NSGs) to different parts of your network. VNet provides the isolation from the internet; subnets provide internal segmentation.

Interview Q: Explain what happens when you delete a Resource Group.

Answer: Deleting a Resource Group permanently deletes ALL resources inside it with no recovery option (unless backups exist in a separate vault). Azure will warn you and ask you to type the resource group name to confirm. If a CanNotDelete lock exists on the resource group, the deletion will fail and you must remove the lock first. This is why production resource groups should always have CanNotDelete locks applied.

Interview Q: What is Azure Bastion and why is it better than opening RDP to the internet?

Answer: Azure Bastion is a managed service that provides RDP and SSH access to VMs through the Azure Portal browser, without needing a public IP on the VM or open RDP/SSH ports. Opening port 3389 (RDP) or 22 (SSH) to the internet is extremely dangerous — automated bots continuously scan the internet for these open ports and attempt brute-force login attacks. Azure Bastion eliminates this attack surface entirely. Authentication uses your Azure AD credentials with MFA. All traffic is TLS-encrypted. Audit logs track every session.

Interview Q: **What are the different ways to achieve high availability in Azure?**

Answer: 1) Availability Sets: Spread VMs across fault domains and upgrade domains in one datacenter. SLA 99.95%. 2) Availability Zones: Spread VMs across physically separate datacenters in the same region. SLA 99.99%. 3) VM Scale Sets (VMSS): Automatically manage multiple identical VMs with autoscaling and load balancing, optionally across zones. 4) Azure Backup + Site Recovery: Data protection and cross-region disaster recovery. 5) Multi-region active-active: Run your app in two or more regions simultaneously behind Traffic Manager or Front Door — the highest level of availability.

Cloud is not the future.

Cloud is the present.

You now have the knowledge to work with enterprise Azure infrastructure.

Build. Break. Fix. Learn. Repeat.

Share this guide freely — knowledge grows when it is shared.

Microsoft Azure Administrator | Free Learning Resource | 2025